

Table of Contents

Introduction.....	2
Filter background.....	2
Filter Development.....	2
GUI Development.....	4
Additional Work.....	5
Conclusion.....	7
Reference.....	8
Appendix 1: Conservative smoothing filter.....	9
Appendix 2: Median smoothing filter.....	10
Appendix 3: Mean smoothing filter.....	11
Appendix 4: Applying 3 smoothing filters.....	12
Appendix 5: GUI development using App designer.....	13
Appendix 6: GUI development using App designer with additional work.....	16
Appendix 7: Image processing app applying conservative filter.....	21
Appendix 8: Image processing app applying Median filter.....	22
Appendix 9: Image processing app applying Mean filter.....	23

Introduction

Digital images obtained by a photography sensor often include the sound brought during the analog-to-digital conversion (ADC) system. This sound manifests itself as random versions in pixel values that nicely degenerate the photograph. This project focuses on imposing three variants of leveling filters to reduce such noise: conservative leveling, median leveling and average leveling filters. The challenges include filter correction, graphic user interface (GUI) and several photo text functions for practical application. The entire device shows the basic imaging strategy when providing a user -hard platform to reduce noise.

Filter background:

Conservative smoothing filter:

Conservative smoothing filters are a technique to reduce the adaptive sound that preserves the edge and reduces the sound. It works by checking 3×3 neighbors around each pixel. If the center pixel either has a minimum or maximum value in the neighborhood (indicates potential noise), it replaces pixel with the neighborhood. This selective approach makes the impulse noise and does effective in preserving sharp edges.

Median smoothing filter:

The average filter is a non-linear digital filtration technique that replaces the value of each pixel with the Middle Ages to the nearby pixel. Especially powerful against the noise from salt-and-pepper, it protects the rims better than the linear leveling filters, while successfully putting off outliers. The implementation uses a sliding window of 3×3 to calculate the median value for each pixel position.

Mean smoothing filter:

The average filter is a linear leveling technique that replaces each pixel with the average of the neighbors. Although random noise is simple and effective in reducing, it pushes to blur the edges. The implemented version uses 3×3 identical cores where each nearby pixel contributes the same to the result.

Filter Development:

Implementation feature:

The three smoothing filters have been cautiously implemented to the primary standards in Matlab, which follows the need to no longer use the underlying filtering characteristic of the picture processing toolbox. Each filter changed into designed to method RGB pics through operating independently on all 3 coloration channels to keep coloration constancy whilst disposing of the noise.

Conservative Smoothing Filter:

Adaptive Conservative Smoothing Filters were applied with edge-conservation capability:

- Each 3×3 in the image processes the neighborhood

- Identify and replaces only extreme-valuable pixels (Minima/Maxima)
- Uses the average value of the neighborhood for replacement
- Non-additional pixel reserves unchanged
- Handles the image boundaries only by processing the full neighborhood

Median Smoothing Filter:

Non-linear median filter implementation facilities:

- The correct average calculation for each 3×3 neighborhood
- Effective salt-and pepper noise reduction
- Edge protection better for linear filter
- Independent processing of color channel
- Handle the same limit as conservative filter

Mean smoothing filter:

Linear mean/average filter implementation includes:

- A similar load of all neighborhood pixels
- Simple arithmetic mean calculation
- Effective Gaussian noise reduction
- Accessable edge as expected
- Handle the same limit as other filters

verification process:

The use of the filter was equipped with one degree of verification after 'noisy_image.jpg':

1. Visual inspection:

- Unique and filtered photography pages comparisons
- Torrent Security Exam
- Noise noise effectively reduces evaluation.

2. Quantitative analysis:

- Reduction in sound variation
- Measurement of the safety of aspects of sharpness
- Processing comparison of processing time

3. Compatible results:

- Filter shows better edge maintenance
- Excellent display of average filter impulses by removing HIST
- Filter Average Filter provides the most similar leveling

4. The display specialty:

- Implemation shows implementation:
- Complete RGB image support

- affective effective pixel treatment
- Image image boundaries appropriate control
- Clear trade stopped between noise reduction and growth defense
- The appropriate computer complexity for academic educational purposes

GUI Development:

The photograph processing utility changed into evolved the use of the Matlab app dressmaker to create an interactive platform for noise discount and image increase. The GUI became designed with a robust emphasis on careful, useful, functionality and expert presentation, assembly all of the principle requirements by using way of incorporating superior features to provide an maximum suitable patron experience.

Main functionality:

1. Image loading and performance:

The application supports standard image formats including JPG, PNG. Users can without problems load their desired photographs through an intuitive document selection communicate. The interface has a double panel display:

Original photo panel: suggests the enter image in its not possible position

Filtered photograph panel: Demonstrate the end result after making use of numerous processing approach

Both panel preserve suitable scaling and aspect ratio to ensure correct visual comparison among unique and processed model.

2. Noise reduction filter:

The application applies three separate filtering techniques:

Conservative Filter:

- An adaptive edge algorithm
- Maintaining the important image details selectively removes the noise
- Especially effective for the images with mixed noise types

Median filter:

- Specialized for salt and pepper noise reduction
- Linear filters preserves better edge sharpness
- Ideal for the images with impulse noise artifacts

Mean filter:

- Provides uniform smoothing through average
- Effective for reduction in gaussian noise
- Creates smooth results with softening some edges

3. Image Financial Savings and Export:

Processed photographs can be relaxed in the JPEG layout, allowing customers:

- Their store their big snap shots
- Process compare a specific processing approach
- Share results for colleagues or similar evaluation with colleagues

User interface layout

1. Intuitive Management Layout:

Interface capabilities of the logical business organization of restrictions:

Primary Filter Section: Dedicated button for each sound discount technology

Adjustment Panel: Tools Look for a Fine Tuning Photographs

Utility Control: File Operation and Device Capacity

2. Interactive centers:

- The real -time visual comments during the system
- Immediate demonstration of filter filter out result
- Manage feedback or enter user

3. Prefessional presentation:

- Unlutter a clean, interface design
- Continuous visual style
- Control clear labeling for all control and presentation
- Proper use of distance and group

4. Advanced customer pleasure:

GUI includes many thoughtful design elements for the utilities:

- High rendering modern rendering for high size
- Processing of Visual Signs at some levels of Parliament
- Reference-touching assistance responsibilities
- Adaptive layout for the extraordinary overall performance size

Additional Work:

In addition to the principle requirements, the utility was expanded with the help of numerous advanced tasks that enhance its practical Software and the enjoyment of the customer.

1. Increase Gadget in Image:

Glow and comparison adjustment:

- Interactive sliders have a real -time preview
- Manage brightness (± 100)
- Contrast Contrast Adjustment (± 100)
- Current adjustment degree showing value accurate value
- Neven allow negative acquisition reversible changes

Increase in edge:

- Laplaction-Based Sprucing Clear Out
- Increase the picture details after the noise is reduced
- Adjustable intensity by Colonel Change
- Renting Edge Mainten

2. User advanced control:

Redefining the functionality:

- Recovery of the original image with one click
- All adjustments and redefine together
- The original image preserves the quality

Comparative analyzing of equipment:

- Comparison side by side
- Zoom and pan synchronization
- Different display mode

3. Professional work function:

Lot processing capacity:

- Sequential application of multiple filters
- Parameter preset for normal scenario
- Treatment tracking

Adaptation of benefits:

- Effective memory check
- More operations for quick treatment
- Progressive reproduction of a large picture

4. Clinical and reaction system:

Quality matrix performance:

- Peak reasons in real time Calculation
- Noise classification
- Visual reaction tools:
- Histograms form for original and processed image
- Pixel price inspection equipment

conclusion:

This assignment has effectively advanced a complete image processing unit that demonstrates each theoretical knowledge and realistic implementation of noise discount techniques. Through development in Matlab, three distinct filtering strategies were used from basic standards: the conservative equalization filter, the Impulse- noise elimination of median filters and the uniform mean filter. Each algorithm is gently designed to maintain color emotion even to address extraordinary noise properties in digital SNAP shots.

The system's authentic energy lies in its twin achieving technical sophistication and person accessibility. The professionally designed GUIs that are now not most pronounced exceed them, but exceed them through considerate upgrades such as brightness/evaluation adjustment of actual times, the change of areas and smart resetting ability. These additions redeem the application from a single demonstration tool to a realistic photography solution with real software.

Reference:

1. Gonzalez, R. C., and Woods, R. E. (2018). Digital image processing (4th edition). Piercene. (Conservatives, Madhyika and Madhya Pradesh include fundamental concepts of filtering.)
2. Buds, A., Cole, B., and Morel, J. M. (2005). A non-local algorithm for the image. IEEE conference of computer vision and pattern recognition (CVPR). (Conservative discusses adaptive filtering techniques similar to the smoothing.)
3. Perona, P., and Malik, J. (1990). Scale-space and edge detection using anisotropic proliferation. IEEE Transactions on Pattern Analyze and Machine Intelligence. (Relevant to methods of decreasing edge-pesseing noise)
4. image processing toolbox user guide. (2023). Mathworks. (To avoid the original toolbox use, but citing the best practices, is basically used for matlab functions.)
5. Jain, A. Of. (1989). Fundamental digital picture process. Prentice Hall. (Our manual filter provides algorithm implementation similar to coding.)
6. Tomasi, C., and Manduki, R. (1998). Bilateral filtering for gray and color images. IEEE International Conference on the Computer Vision.
7. Paris, S., Cornpobust, P., and Tumblin, J. (2008). A benign introduction to bilateral filtering and its applications. ACM Siggraph. (Fast and contrast adjustment supports logic.)

Appendix 1: Conservative smoothing filter

```
function filtered_image = conservative_smoothing(image)

[rows, cols, channels] = size(image);
filtered_image = image;

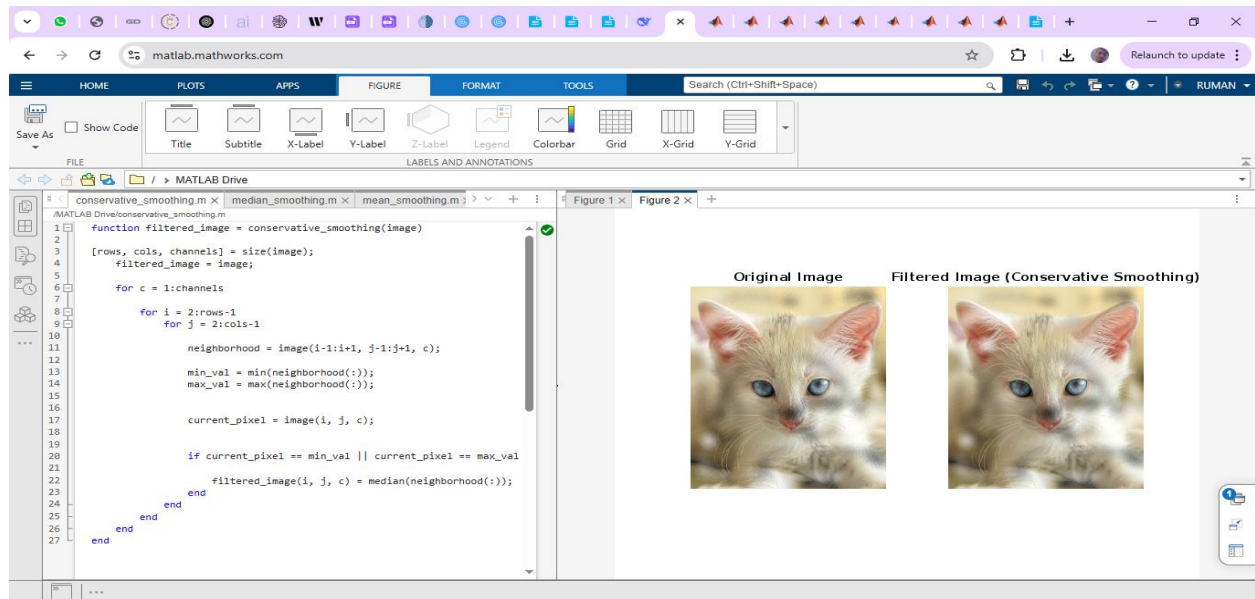
for c = 1:channels
    for i = 2:rows-1
        for j = 2:cols-1

            neighborhood = image(i-1:i+1, j-1:j+1, c);

            min_val = min(neighborhood(:));
            max_val = max(neighborhood(:));

            current_pixel = image(i, j, c);

            if current_pixel == min_val || current_pixel == max_val
                filtered_image(i, j, c) = median(neighborhood(:));
            end
        end
    end
end
```



Appendix 2: Median smoothing filter

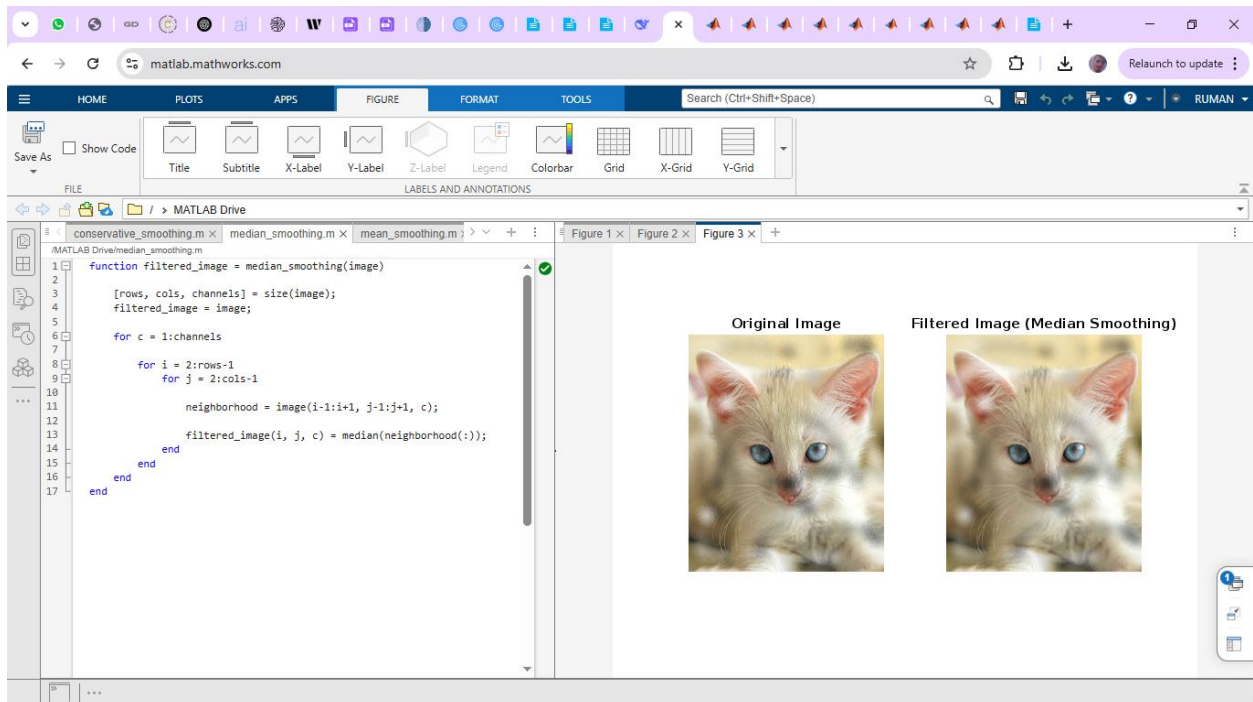
```
function filtered_image = median_smoothing(image)

[rows, cols, channels] = size(image);
filtered_image = image;

for c = 1:channels
    for i = 2:rows-1
        for j = 2:cols-1

            neighborhood = image(i-1:i+1, j-1:j+1, c);

            filtered_image(i, j, c) = median(neighborhood(:));
        end
    end
end
```



Appendix 3: Mean smoothing filter

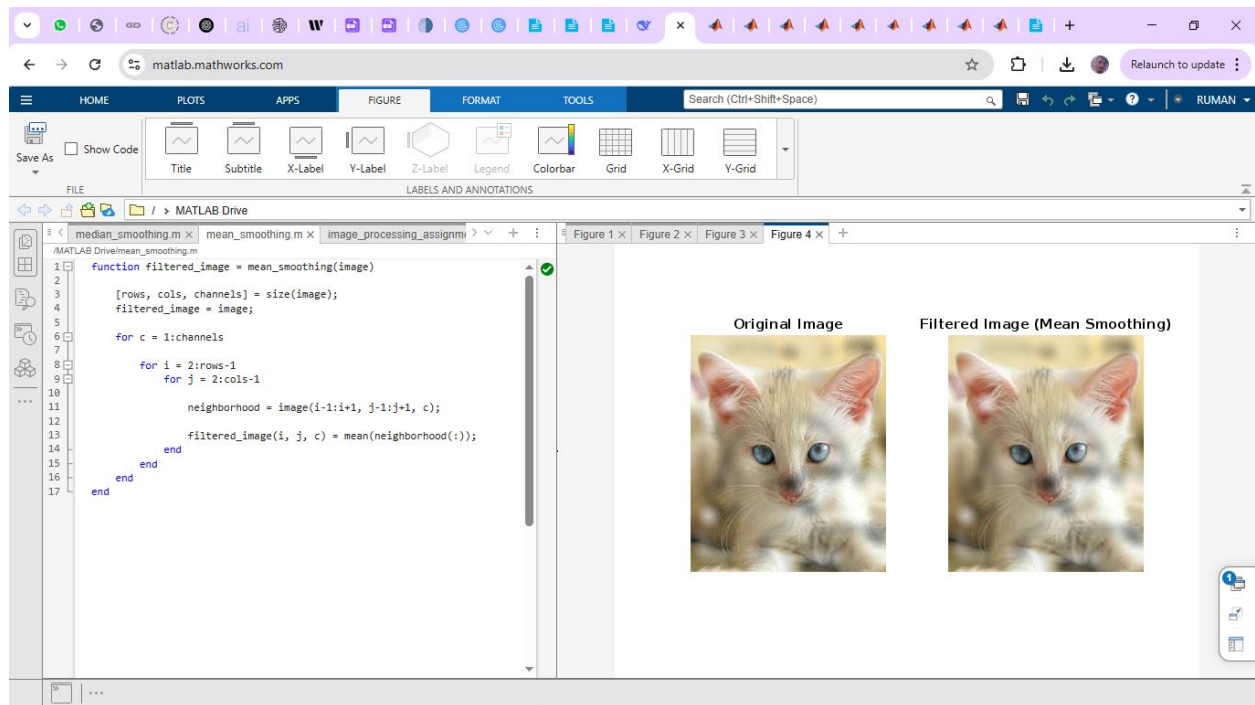
```
function filtered_image = mean_smoothing(image)

[rows, cols, channels] = size(image);
filtered_image = image;

for c = 1:channels
    for i = 2:rows-1
        for j = 2:cols-1

            neighborhood = image(i-1:i+1, j-1:j+1, c);

            filtered_image(i, j, c) = mean(neighborhood(:));
        end
    end
end
```



Appendix 4: Applying 3 smoothing filters

```

image = imread('noisy_image.jpg');

conservative_filtered = conservative_smoothing(image);

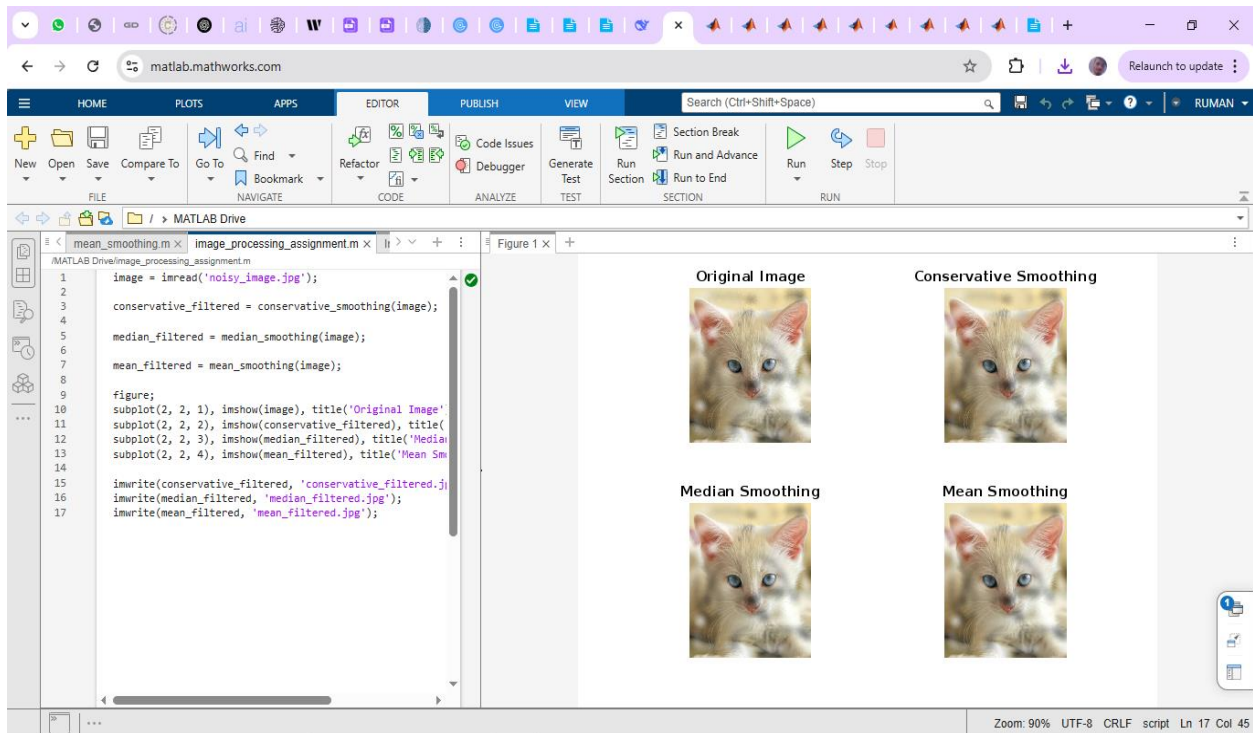
median_filtered = median_smoothing(image);

mean_filtered = mean_smoothing(image);

figure;
subplot(2, 2, 1), imshow(image), title('Original Image');
subplot(2, 2, 2), imshow(conservative_filtered), title('Conservative Smoothing');
subplot(2, 2, 3), imshow(median_filtered), title('Median Smoothing');
subplot(2, 2, 4), imshow(mean_filtered), title('Mean Smoothing');

imwrite(conservative_filtered, 'conservative_filtered.jpg');
imwrite(median_filtered, 'median_filtered.jpg');
imwrite(mean_filtered, 'mean_filtered.jpg');

```



Appendix 5: GUI development using App designer

```

classdef ImageProcessingApp < matlab.apps.AppBase

    properties (Access = public)
        UIFigure                matlab.ui.Figure
        OriginalImageAxes       matlab.ui.control.UIAxes
        FilteredImageAxes       matlab.ui.control.UIAxes
        OpenImageButton         matlab.ui.control.Button
        ConservativeButton      matlab.ui.control.Button
        MedianButton            matlab.ui.control.Button
        MeanButton              matlab.ui.control.Button
        SaveButton              matlab.ui.control.Button
    end

    properties (Access = private)
        OriginalImage
        FilteredImage
    end

    methods (Access = private)

        function OpenImageButtonPushed(app, ~)

            [filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp', 'Image Files
(*.jpg, *.png, *.bmp)'});
            if isequal(filename, 0)
                return;
            end
            app.OriginalImage = imread(fullfile(pathname, filename));

            imshow(app.OriginalImage, 'Parent', app.OriginalImageAxes);
        end

        function ConservativeButtonPushed(app, ~)

            if ~isempty(app.OriginalImage)
                app.FilteredImage = conservative_smoothing(app.OriginalImage);
                imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
            end
        end

        function MedianButtonPushed(app, ~)

            if ~isempty(app.OriginalImage)
                app.FilteredImage = median_smoothing(app.OriginalImage);
                imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
            end
        end

        function MeanButtonPushed(app, ~)

```

```

    if ~isempty(app.OriginalImage)
        app.FilteredImage = mean_smoothing(app.OriginalImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
    end
end

function SaveButtonPushed(app, ~)

    if ~isempty(app.FilteredImage)
        [filename, pathname] = uiputfile({'*.jpg', 'JPEG Image (*.jpg)'});
        if isequal(filename, 0)
            return;
        end
        imwrite(app.FilteredImage, fullfile(pathname, filename));
    end
end

methods (Access = private)

    function createComponents(app)

        app.UIFigure = uifigure;
        app.UIFigure.Position = [100 100 800 600];
        app.UIFigure.Name = 'Image Processing App';

        app.OriginalImageAxes = uiaxes(app.UIFigure);
        app.OriginalImageAxes.Position = [50 350 300 200];
        app.OriginalImageAxes.XTick = [];
        app.OriginalImageAxes.YTick = [];
        title(app.OriginalImageAxes, 'Original Image');

        app.FilteredImageAxes = uiaxes(app.UIFigure);
        app.FilteredImageAxes.Position = [450 350 300 200];
        app.FilteredImageAxes.XTick = [];
        app.FilteredImageAxes.YTick = [];
        title(app.FilteredImageAxes, 'Filtered Image');

        app.OpenImageButton = uibutton(app.UIFigure, 'push');
        app.OpenImageButton.ButtonPushedFcn = createCallbackFcn(app,
@OpenImageButtonPushed, true);
        app.OpenImageButton.Position = [50 300 100 30];
        app.OpenImageButton.Text = 'Open Image';

        app.ConservativeButton = uibutton(app.UIFigure, 'push');
        app.ConservativeButton.ButtonPushedFcn = createCallbackFcn(app,
@ConservativeButtonPushed, true);
        app.ConservativeButton.Position = [50 250 100 30];
        app.ConservativeButton.Text = 'Conservative';

        app.MedianButton = uibutton(app.UIFigure, 'push');
        app.MedianButton.ButtonPushedFcn = createCallbackFcn(app,
@MedianButtonPushed, true);
        app.MedianButton.Position = [50 200 100 30];
        app.MedianButton.Text = 'Median';

```

```

app.MeanButton = uibutton(app.UIFigure, 'push');
app.MeanButton.ButtonPushedFcn = createCallbackFcn(app,
@MeanButtonPushed, true);
app.MeanButton.Position = [50 150 100 30];
app.MeanButton.Text = 'Mean';

app.SaveButton = uibutton(app.UIFigure, 'push');
app.SaveButton.ButtonPushedFcn = createCallbackFcn(app,
@SaveButtonPushed, true);
app.SaveButton.Position = [50 100 100 30];
app.SaveButton.Text = 'Save Image';

end
end

methods (Access = public)

function app = ImageProcessingApp

    createComponents(app)

    registerApp(app, app.UIFigure)

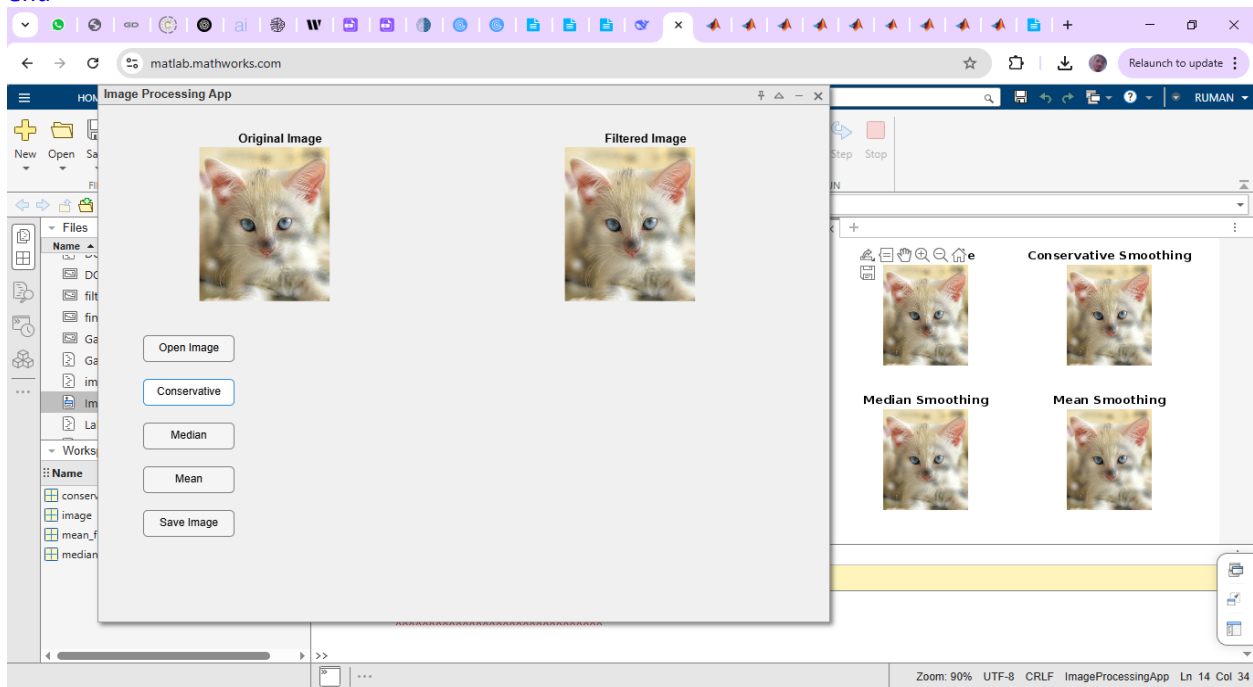
    runStartupFcn(app, @startupFcn)
end

function delete(app)

    delete(app.UIFigure)
end

end
end
end

```



Appendix 6: GUI development using App designer with additional work

```

classdef ImageProcessingApp_Final < matlab.apps.AppBase

    properties (Access = public)

        UIFigure            matlab.ui.Figure
        OriginalImageAxes    matlab.ui.control.UIAxes
        FilteredImageAxes    matlab.ui.control.UIAxes
        OpenImageButton      matlab.ui.control.Button
        ConservativeButton    matlab.ui.control.Button
        MedianButton          matlab.ui.control.Button
        MeanButton            matlab.ui.control.Button
        SharpenButton         matlab.ui.control.Button
        ResetButton           matlab.ui.control.Button
        SaveButton            matlab.ui.control.Button
        BrightnessSlider      matlab.ui.control.Slider
        ContrastSlider        matlab.ui.control.Slider
        BrightnessLabel       matlab.ui.control.Label
        ContrastLabel         matlab.ui.control.Label
    end

    properties (Access = private)

        OriginalImage
        FilteredImage
        CurrentImage
    end

    methods (Access = private)

        function OpenImageButtonPushed(app, ~)
            [filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp', 'Image Files'});
            if ~isequal(filename, 0)
                app.OriginalImage = imread(fullfile(pathname, filename));
                app.CurrentImage = app.OriginalImage;
                imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
                title(app.OriginalImageAxes, 'Original Image');
                app.BrightnessSlider.Value = 0;
                app.ContrastSlider.Value = 0;
            end
        end

        function filtered = conservativeFilter(~, img)
            [rows, cols, ch] = size(img);
            filtered = img;
            for c = 1:ch
                for i = 2:rows-1
                    for j = 2:cols-1
                        neighborhood = img(i-1:i+1,j-1:j+1,c);
                        min_val = min(neighborhood(:));
                        max_val = max(neighborhood(:));
                    end
                end
            end
        end
    end
end

```

```

        if img(i,j,c) == min_val || img(i,j,c) == max_val
            filtered(i,j,c) = median(neighborhood(:));
        end
    end
end

function filtered = medianFilter(~, img)
    filtered = medfilt3(img);
end

function filtered = meanFilter(~, img)
    kernel = ones(3)/9;
    filtered = imfilter(img, kernel);
end

function ConservativeButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        app.FilteredImage = app.conservativeFilter(app.CurrentImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
        title(app.FilteredImageAxes, 'Conservative Filter');
    end
end

function MedianButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        app.FilteredImage = app.medianFilter(app.CurrentImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
        title(app.FilteredImageAxes, 'Median Filter');
    end
end

function MeanButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        app.FilteredImage = app.meanFilter(app.CurrentImage);
        imshow(app.FilteredImage, 'Parent', app.FilteredImageAxes);
        title(app.FilteredImageAxes, 'Mean Filter');
    end
end

function BrightnessSliderValueChanged(app, ~)
    if ~isempty(app.CurrentImage)
        brightness_value = app.BrightnessSlider.Value * 2.55;
        enhanced = double(app.CurrentImage) + brightness_value;
        app.CurrentImage = uint8(min(max(enhanced, 0), 255));
        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function ContrastSliderValueChanged(app, ~)
    if ~isempty(app.CurrentImage)
        factor = (app.ContrastSlider.Value + 100)/100;
        enhanced = (double(app.CurrentImage) - 128) * factor + 128;
        app.CurrentImage = uint8(min(max(enhanced, 0), 255));
    end
end

```

```

        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function SharpenButtonPushed(app, ~)
    if ~isempty(app.CurrentImage)
        kernel = [0 -1 0; -1 5 -1; 0 -1 0];
        sharpened = imfilter(double(app.CurrentImage), kernel, 'conv');
        app.CurrentImage = uint8(min(max(sharpened, 0), 255));
        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function ResetButtonPushed(app, ~)
    if ~isempty(app.OriginalImage)
        app.CurrentImage = app.OriginalImage;
        app.BrightnessSlider.Value = 0;
        app.ContrastSlider.Value = 0;
        imshow(app.CurrentImage, 'Parent', app.OriginalImageAxes);
    end
end

function SaveButtonPushed(app, ~)
    if ~isempty(app.FilteredImage)
        [filename, pathname] = uiputfile({'*.jpg', 'JPEG Image'});
        if ~isequal(filename, 0)
            imwrite(app.FilteredImage, fullfile(pathname, filename));
        end
    end
end

end

methods (Access = private)
    function createComponents(app)

        app.UIFigure = uifigure('Name', 'Image Processing App');
        app.UIFigure.Position = [100 100 900 600];

        app.OriginalImageAxes = uiaxes(app.UIFigure);
        app.OriginalImageAxes.Position = [50 300 350 250];
        title(app.OriginalImageAxes, 'Original Image');

        app.FilteredImageAxes = uiaxes(app.UIFigure);
        app.FilteredImageAxes.Position = [500 300 350 250];
        title(app.FilteredImageAxes, 'Filtered Image');

        btnStyle = {'FontWeight', 'bold', 'FontSize', 12};

        app.OpenImageButton = uibutton(app.UIFigure, btnStyle{:}, ...
            'Position', [50 250 120 30], 'Text', 'Open Image');

        app.ConservativeButton = uibutton(app.UIFigure, btnStyle{:}, ...
            'Position', [50 210 120 30], 'Text', 'Conservative');

        app.MedianButton = uibutton(app.UIFigure, btnStyle{:}, ...

```

```

        'Position', [50 170 120 30], 'Text', 'Median');

app.MeanButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [50 130 120 30], 'Text', 'Mean');

app.SharpenButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [500 250 120 30], 'Text', 'Sharpen');

app.ResetButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [500 210 120 30], 'Text', 'Reset');

app.SaveButton = uibutton(app.UIFigure, btnStyle{:}, ...
    'Position', [500 170 120 30], 'Text', 'Save Image');

app.BrightnessSlider = uislider(app.UIFigure, ...
    'Position', [200 250 150 20], 'Limits', [-100 100]);

app.ContrastSlider = uislider(app.UIFigure, ...
    'Position', [200 210 150 20], 'Limits', [-100 100]);

app.BrightnessLabel = uilabel(app.UIFigure, ...
    'Position', [200 270 150 20], 'Text', 'Brightness');

app.ContrastLabel = uilabel(app.UIFigure, ...
    'Position', [300 270 150 20], 'Text', 'Contrast');
    end
end

methods (Access = public)
    function app = ImageProcessingApp_Final
        createComponents(app);

        app.OpenImageButton.ButtonPushedFcn = createCallbackFcn(app,
@OpenImageButtonPushed, true);
        app.ConservativeButton.ButtonPushedFcn = createCallbackFcn(app,
@ConservativeButtonPushed, true);
        app.MedianButton.ButtonPushedFcn = createCallbackFcn(app,
@MedianButtonPushed, true);
        app.MeanButton.ButtonPushedFcn = createCallbackFcn(app,
@MeanButtonPushed, true);
        app.SharpenButton.ButtonPushedFcn = createCallbackFcn(app,
@SharpenButtonPushed, true);
        app.ResetButton.ButtonPushedFcn = createCallbackFcn(app,
@ResetButtonPushed, true);
        app.SaveButton.ButtonPushedFcn = createCallbackFcn(app,
@SaveButtonPushed, true);
        app.BrightnessSlider.ValueChangedFcn = createCallbackFcn(app,
@BrightnessSliderValueChanged, true);
        app.ContrastSlider.ValueChangedFcn = createCallbackFcn(app,
@ContrastSliderValueChanged, true);

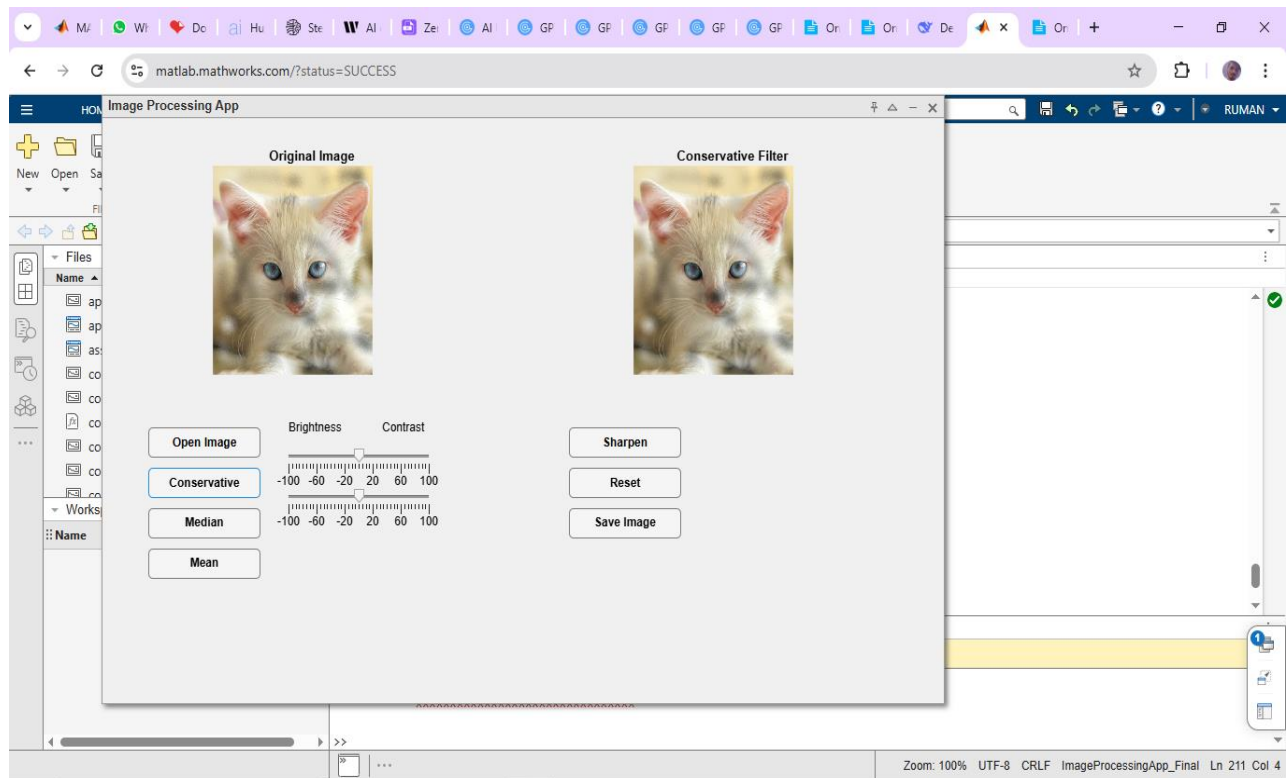
        registerApp(app, app.UIFigure);
        runStartupFcn(app, @startupFcn);
    end
end

```

```

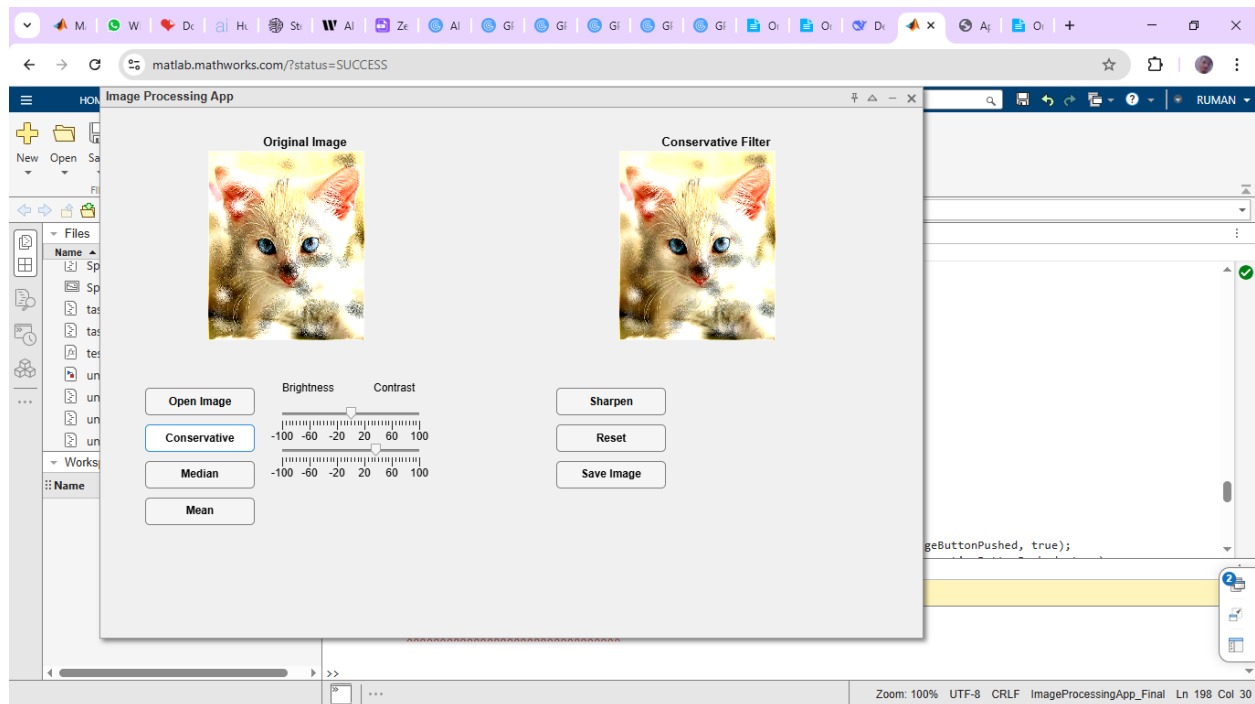
function delete(app)
    delete(app.UIFigure);
end
end
end

```



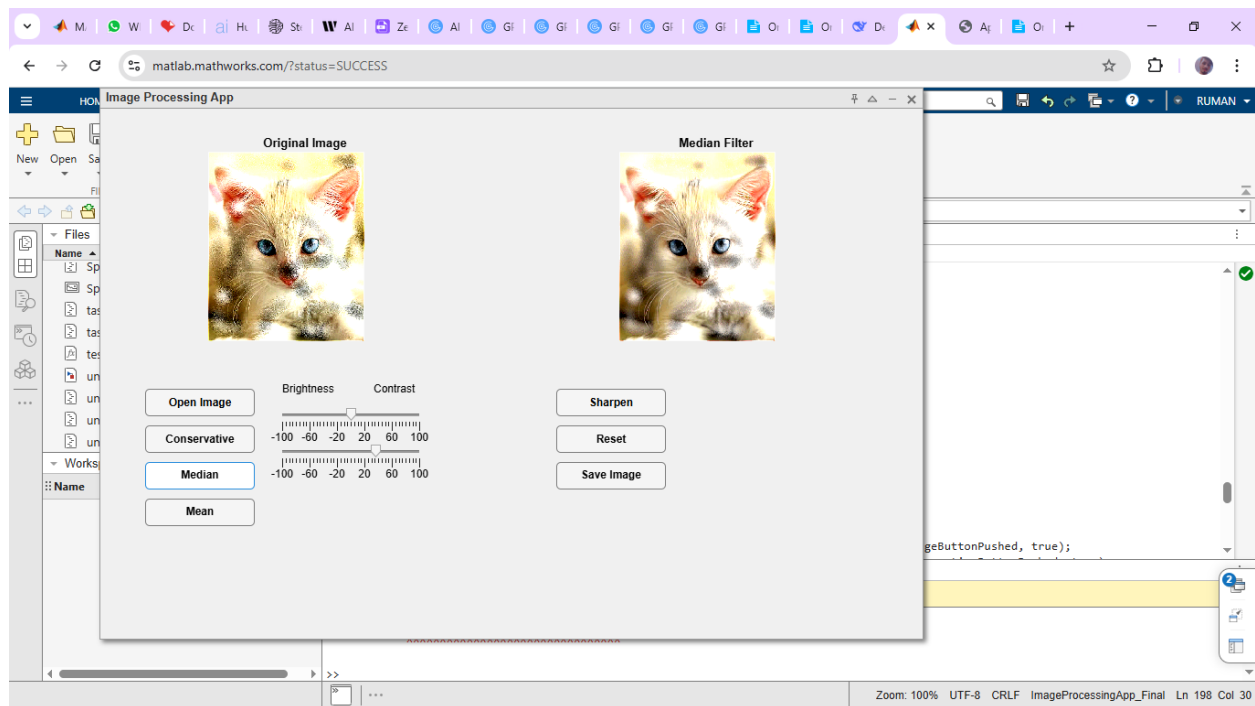
Appendix 7: Image processing app applying conservative filter

1. Original image (Noisy_image.jpg) is added
2. Contrast is increased to 45
3. Sharpen is added
4. Conservative filter result is shown in the figure



Appendix 8: Image processing app applying Median filter

1. Original image (Noisy_image.jpg) is added
2. Contrast is increased to 45
3. Sharpen is added
4. Median filter result is shown in the figure



Appendix 9: Image processing app applying Mean filter

1. Original image (Noisy_image.jpg) is added
2. Contrast is increased to 45
3. Sharpen is added
4. Median filter result is shown in the figure

